

PalmSource®

Conference and Expo



Incorporating Animation & Graphics

Howard Tomlinson, CEO Astraware

David Oakley, CTO Astraware



Palm Powered™ handhelds can come to life by using animation, but there are some tricks you need to know. This session gives you the heads up.



Incorporating Animation & Graphics

- A background to animation
- Graphics techniques
- Device Issues
- Compatibility and Optimization



What is animation?

- Animated cartoon (Merriam-Webster, 1915)
  - “a motion picture made from a series of drawings simulating motion by means of slight progressive changes in the drawings”
- Speed
  - About 30 frames (drawings) per second (FPS) is required to fool the brain that motion is continuous
- Progression
  - The changes have to be big enough to see, without being so large that the jumps are unrealistic



Limitations of Palm OS® devices

- Operating system support
  - Very simple graphic primitives only
  - No circles, triangles, polygons
  - No 3D
  - Compatibility over speed
- Device limitations
  - Small, low-resolution screen
  - Persistent screen on greyscale devices (a plus?)
  - Slow processor (relatively)
- OK for business applications

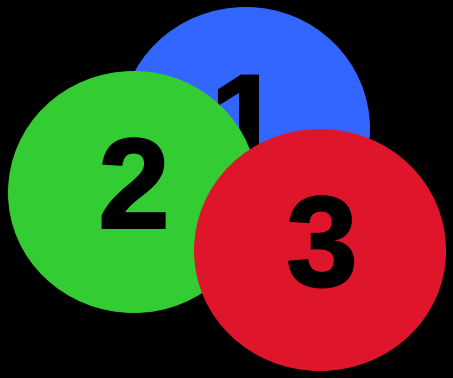


Incorporating Animation & Graphics

- A background to animation
- Graphics techniques
- Device Issues
- Compatibility and Optimization

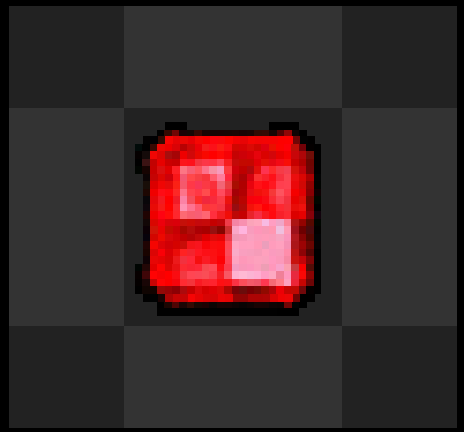
What is a sprite and how are they created?

- A sprite is a discrete graphical image that can move on the screen
- Sprites often have a ‘z-index’, determining which is in front when several sprites overlap
  - The Painter’s Algorithm



What is a sprite and how are they created?

- A simple, rectangular sprite can be drawn using **`WinCopyRectangle`**…
- …but usually the sprite has an irregular outline requiring a ‘mask’ which allows the background to show through





Animation in your application code

- Initialisation code
  - Load the graphics you'll need in your animation
  - Set up the positions of your sprites
  - Store the value of **`TimGetTicks`**, which will be used to periodically 'fire' new frames
- In event loop
  - Use **`EvtGetEvent`** with a timeout of 0 or 1 ticks
  - When any event is fired (inc. **`nilEvent`**s), check whether it is time to start a new frame
  - Refresh screen if a new frame is required

Be prepared (to go faster)

- Business applications tend to use **`WinDrawBitmap`** throughout
  - This is a slow function, esp. on compressed bitmaps
  - Very little flexibility
- Programs using animation should only use this to load off-screen windows on start-up (using **`WinCreateOffscreenWindow`**)
- Once the off-screen window has been loaded, application can then use **`WinCopyRectangle`**
  - Fast
  - Many drawing options, for masks, etc.

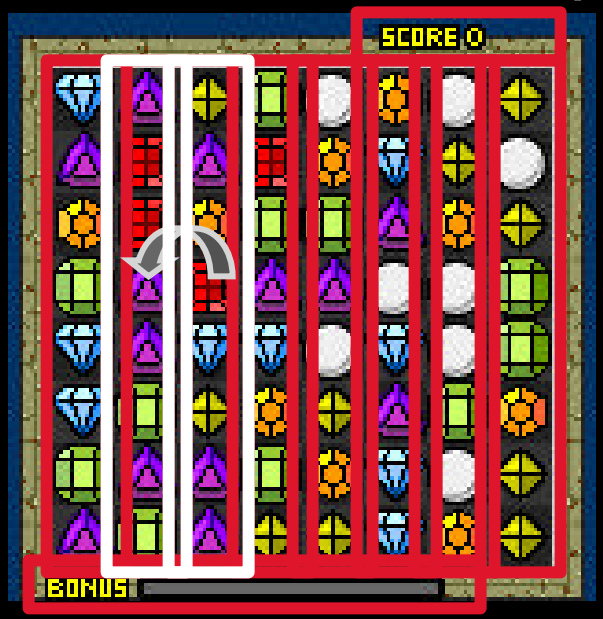


Building up a frame

- Easiest way – redraw the screen completely
  - Wipe it blank
  - Draw each sprite from furthest to nearest
  - Easy – but lots of redundant drawing if not all the sprites move
- Harder way – only redraw the changes
  - Work out which areas have changed
  - Update each section that has changed
  - Potentially faster, as less graphics drawn, but extra overhead to keep track of changes
- Trade-off - which is appropriate…

Example: Bejeweled

- Screen is split into components
  - 10 vertical strips of gems, score area, bonus bar
- Algorithm determines whether it's cheaper to redraw whole screen or components





Incorporating Animation & Graphics

- A background to animation
- Graphics techniques
- Device Issues
- Compatibility and Optimization



Color tables

- Use color tables in 8-bit color mode if you need colors that aren't in the standard "web safe" set
- Two options for using color tables
  - Replace the whole palette (remember UI elements)
  - Use the unused 16 entries at the top of the palette
- Use **`WinPalette`** to get the old palette, set up the new one
  - Options to change individual or all entries
- Color tables are slow! Don't use them repeatedly.



Supporting Different Resolutions

- Monochrome
- 2 bit Greyscale
- 4 bit Greyscale
- 8 bit Color
- 16 bit Color
- Separate prc files, or all together?
  - Size vs Convenience

16-bit Color

- Each word in screen memory represents 5 bits of red, 6 of green, 5 of blue
- Beware of heap space requirements
  - Mask planes will be 16 bit too
  - Twice the storage capacity compared to 8-bit
- How much heap space to take up?
  - If using your own routines you could store static images in feature or database memory





Multi-resolutions in one program

- Bitmap Families – let the OS work for you!
- Store resources in ‘add on’ databases.
- If you’re “doing it yourself” check that the device can handle the resolution you want.
- All bitmaps you use will be expanded to the current resolution of the device. (Beware heap usage!)



Devices and speeds

Device	Screen	Depth	Speed	Kbits/MHz
Palm™ III	160x160	2	16	3200
Palm™ Vx	160x160	4	20	5120
Palm™ IIIc	160x160	8	20	10240
Visor Prism Palm™ m505	160x160	16	33	12412
Visor Platinum Palm™ m500	160x160	4	33	3103
Sony Clié 760C	320x320	16	33	49648



Incorporating Animation & Graphics

- A background to animation
- Graphics techniques
- Device Issues
- Compatibility and Optimization



Animating the Palm OS legal way

- Use **`WinCopyRectangle`** over and over…
  - To draw background and images to buffer
  - To copy buffer to screen
  - And it even knows how to do it for different depths!
- You *should* get sufficient performance to convince people
  - Providing you aren't trying to move too many graphics around per frame
- Benefits of stopping here
  - Palm approval
  - Device independence – only one build required

Breaking the law

- Palm OS was written for device independence
  - Written in C for generic hardware…
  - …for the lowest common denominator
- You know your app better than Palm does!
  - Special case – copying the whole screen
  - Cut corners that Palm wouldn't
- Access the screen directly
  - It is just a chunk of memory (at the moment)
  - use MemMove or your own C or 68000
- Detect hardware / OS, and degrade gracefully

Other optimization options

- Write your own ‘blitting’ code
  - Specifically for 1,2,4,8,16 bit depth sprite drawing
  - Writing direct to off-screen buffers is legal
- Understand the bottlenecks
  - Where does your App spend most of its time?
  - Apps often spend 90% of their time in 10% of the code – optimize that 10% first
- Pre-generate as much as possible
  - Such as sprites with several layers
  - But bear in mind memory restrictions

New devices, new challenges

- Sony Clié
  - 320x320 = 4 times the screen area
  - Accelerator chip for fast access
- Increasing number of 16-bit color devices
- Palm OS 5
  - Current 68000 apps are emulated (relatively slowly)
  - OS routines in native ARM (faster)
  - These devices will probably also have large screens
  - You must use **`TimGetTicks()`** to get the timing right



Getting Help

- Palm OS Website
  - Knowledge base has useful articles
  - Developer forum
  - http://www.palmos.com/
- Razor! Framework
  - Includes routines for handling sprites and a “canvas”
  - Optimised drawing routines
  - Various other routines to help create an animated and interactive application
  - http://www.tilo-christ.de/razor/



Summary

- Prepare your graphics on start-up
- Optimize for speed…
- …but limit your speed on fast processors
- Stick to the Palm OS® routines if at all possible



Thank you!

- Q&A



www.astraware.com



PalmSource®

Conference and Expo